

专业 计算机科学与技术 班级 计科 20208 班 成绩

学号 20201008303 姓名 王韵菲

实验八 ARM 的中断实验

一、 实验目的

1. 掌握 ARM9 的中断原理, 能够对 S3C2410 的中断资源及其相关中断寄存器的进行合理配置

2. 掌握对 S3C2410 的中断的编程的方法

二、 实验环境

1. EL-ARM-860 教学实验箱, PentiumII 以上的 PC 机, 仿真调试电缆。

2. PC 操作系统 WIN98 或 WIN2000 或 WINXP, ADS1.2 集成开发环境, 仿真调试驱动程序。

三、 实验内容

具体内容:

学习响应外部中断请求的配置方法, 并通过响应定时器中断, 执行中断服务子程序使 CPU 板上的 LED 指示灯 LED1、LED2 闪烁。

实验步骤:

1. 本实验仅使用实验教学系统的核心 CPU 板。在进行本实验时, LCD 电源开关, 音频的左右声道开关、AD 通道选择开关、触摸屏中断选择开关等均应处在关闭状态。

2. 在 PC 机并口和实验箱的 CPU 板上的 JTAG 接口之间, 连接仿

真调试电缆以及串口间连接公/母接头串口线。

3. 打开超级终端，配置串口的属性（如 COM1），配置波特率为 115200，校验位无，数据位为 8，停止位为 1，数据控制流为无；检查连接是否可靠，可靠后，接入电源线，系统上电，同时按住“空格”键，进入 VIVI 状态。

4. 打开 ADS1.2 开发环境，从里面打开\实验程序\HARDWARE\ADS\实验六\Interrupt.mcp 项目文件，进行编译。

5. 编译通过后，进入 ADS1.2 调试界面，加载\实验程序\HARDWARE\ADS\实验六\Interrupt_Data\Debug 中的映像文件程序映像 Interrupt.axf。

6. 在 ADS 调试环境下全速运行映像文件。观察 LED1 和 LED2 的变化！LED1 和 LED2 灯会由于定时中断的 1 秒钟发生一次，而一秒钟闪烁一次！也可以改变闪烁的频率，即改变 Startup2410\target.c 文件内的 void Timer1_init(void) 函数里的 rTCNTB1 = 48828; 的赋值，数字量越小，闪烁频率越快。编译全速运行，观看结果，看闪烁频率是否发生了改变！这是对 GPIO 口操作的结果。具体实现见程序。

(1) 异常及中断控制器的初始化:

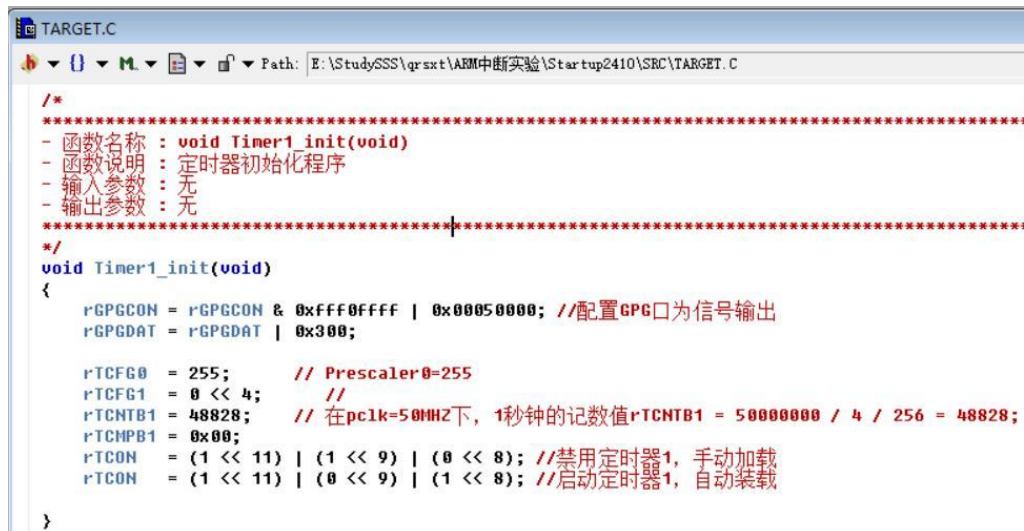


```
/*
*****
- 函数名称 : Exep_S3cint_Init(void)
- 函数说明 : 异常及中断控制器的初始化
- 输入参数 : 无
- 输出参数 : 无
*****
*/
void Isr_Init(void)
{
    pISR_UNDEF = (unsigned)HaltUndef;
    pISR_SWI   = (unsigned)HaltSwi;
    pISR_PABORT = (unsigned)HaltPabort;
    pISR_DABORT = (unsigned)HaltDabort;

    rINTMOD    = 0x0;           //All=IRQ mode
    rINTMSK    = BIT_ALLMSK;   //All interrupt is masked.
    rINTSUBMSK = BIT_SUB_ALLMSK; //All sub-interrupt is masked. <- April 01, 2002 SOP
}
```

图 1 异常及中断控制器的初始化程序

(2) 定时器初始化程序:

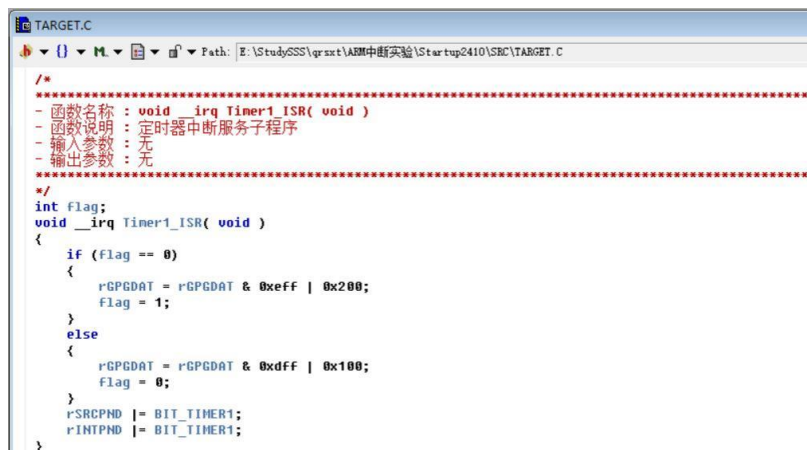


```
/*
*****
- 函数名称 : void Timer1_init(void)
- 函数说明 : 定时器初始化程序
- 输入参数 : 无
- 输出参数 : 无
*****
*/
void Timer1_init(void)
{
    rGPGCON = rGPGCON & 0xffff0ffff | 0x00050000; //配置GPG口为信号输出
    rGPGDAT = rGPGDAT | 0x300;

    rTCFG0 = 255; // Prescaler=255
    rTCFG1 = 0 << 4; //
    rTCNTB1 = 48828; // 在pclk=50MHZ下, 1秒钟的记数值rTCNTB1 = 50000000 / 4 / 256 = 48828;
    rTCMPB1 = 0x00;
    rTCON = (1 << 11) | (1 << 9) | (0 << 8); //禁用定时器1, 手动加载
    rTCON = (1 << 11) | (0 << 9) | (1 << 8); //启动定时器1, 自动装载
}
```

图 2 定时器初始化程序

(3) 定时器中断服务子程序:



```
/*
*****
- 函数名称 : void __irq Timer1_ISR( void )
- 函数说明 : 定时器中断服务子程序
- 输入参数 : 无
- 输出参数 : 无
*****
*/
int flag;
void __irq Timer1_ISR( void )
{
    if (flag == 0)
    {
        rGPGDAT = rGPGDAT & 0xe0ff | 0x200;
        flag = 1;
    }
    else
    {
        rGPGDAT = rGPGDAT & 0xdff | 0x100;
        flag = 0;
    }
    rSRCPND |= BIT_TIMER1;
    rINTPND |= BIT_TIMER1;
}
```

图 3 定时器中断服务子程序

(4) 定时器中断初始化子程序：

```
TARGET.C
Path: E:\StudySSS\qrsxt\ARM中断实验\Startup2410\Src\TARGET.C

/*
*****
- 函数名称 : void Timer1INT_Init(void)
- 函数说明 : 定时器中断初始化程序
- 输入参数 : 无
- 输出参数 : 无
*****
*/
void Timer1INT_Init(void)                                //定时器接口使能
{
    if ((rINTPND & BIT_TIMER1))
    {
        rSRCPND |= BIT_TIMER1;
    }
    pISR_TIMER1 = (int)Timer1_ISR;
    rINTMSK &= ~(BIT_TIMER1); //开中断;
}
```

图 4 定时器中断初始化子程序

(5) 目标板初始化程序, 主函数中只需调用此函数, 即可完成目标板的初始化

```
TARGET.C
Path: E:\StudySSS\qrsxt\ARM中断实验\Startup2410\Src\TARGET.C

/*
*****
- 函数名称 : TargetInit(void)
- 函数说明 : 目标板初始化程序,主函数中只需调用此函数, 即可完成目标板的初始化
- 输入参数 : 无
- 输出参数 : 无
*****
*/
void Target_Init(void)
{
    MMIO_Init();
    ChangeClockDivider(1,1); // 1:2:4
    ChangeMPLLValue(0xa1,0x3,0x1); // FCLK=202.8MHz
    Isr_Init();
    Port_Init();

    Timer1_init();
    Timer1INT_Init();

    Uart_Init(0,115200);
    Uart_Select(0);
}
```

图 5 目标板初始化程序

通过实验, 改变定时器初始化程序的 $rTCNTB1 = 48828$ 的值为 24414, 可以发现值越小, 闪烁频率越快。

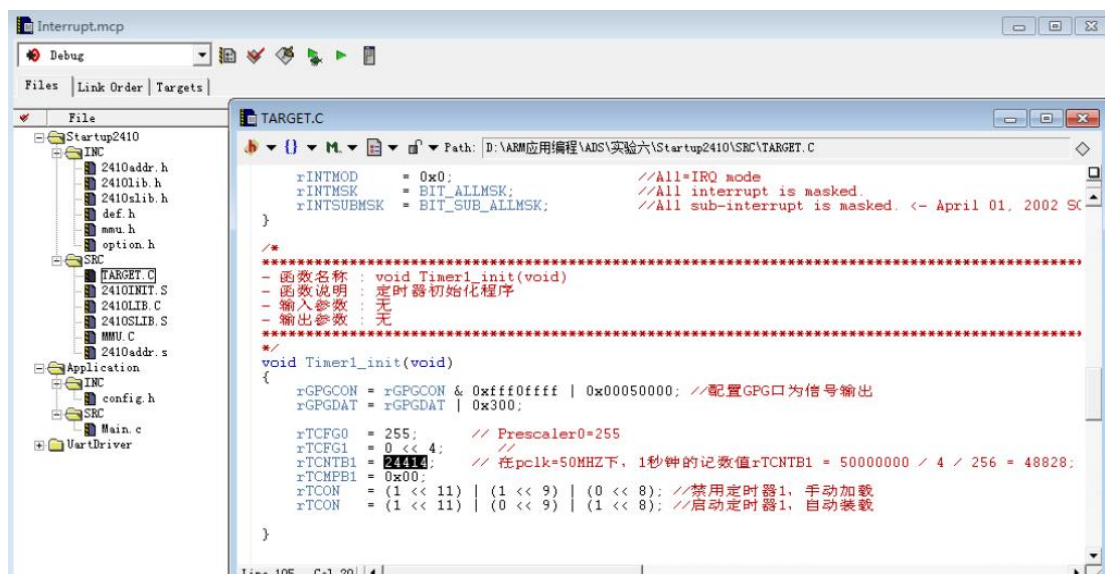


图 1 运行程序图

四、 实验心得

1、 写出自己的收获

(1) 中断请求的配置方法：学习了如何配置外部中断请求。这包括了连接中断源到正确的引脚,配置中断触发方式(如上升沿触发、下降沿触发等)，以及使能相应的中断控制器。

(2) 定时器中断的工作原理：理解了定时器中断的原理和作用。定时器可以周期性地生成中断请求,通过配置定时器的计数值和时钟源,可以控制定时器中断的触发频率。

(3) 中断服务子程序的执行：学会编写中断服务子程序（ISR）来响应中断请求。中断服务子程序是在中断触发时由系统自动调用的函数,可以在其中执行特定的操作,比如控制 LED 指示灯的闪烁。

(4) GPIO 口控制 LED 指示灯：熟悉了在嵌入式系统中使用 GPIO 口控制 LED 指示灯的方法。通过配置 GPIO 口的输入输出状态和电平,可以实现对 LED 的开关控制。

2、实验过程中出现的问题

如果 LED1 和 LED2 没有按预期闪烁,可能是定时器中断配置或中断服务子程序的问题。